Comment Category Prediction

Multi-Class NLP Classification

Technical Report

1. Project Overview

This project addresses a multi-class text classification problem: given a user comment and its associated metadata, predict which of four categories (labels 0–3) the comment belongs to. The dataset comprises 198,000 training records and 102,000 test records across 15 columns, combining raw comment text with community engagement signals (upvotes, downvotes), emoticon usage, demographic metadata (race, religion, gender), and post/timestamp information.

The solution is built on a pipeline of text preprocessing, over 20 hand-crafted features, dual-level TF-IDF vectorization producing 60,036 total features, and a LightGBM gradient boosting classifier — achieving 96.9% validation accuracy and a weighted F1-score of 0.9698 on the final tuned model.

Column	Description
comment	Raw text of the user comment — the primary modelling signal
upvote / downvote	Community engagement counts. Upvote mean=2.61, downvote mean=0.67 (right-skewed)
emoticon_1 / 2 / 3	Emoticon frequency counts. emoticon_1 has max value 46; emoticon_2 max 11; emoticon_3 max 17
race / religion / gender	Demographic metadata. 145,423 of 198,000 records have NaN in all three — handled explicitly
post_id	Thread identifier (range 20–119); used for post-level aggregation features
created_date	Timezone-aware timestamp; parsed to extract hour, dayofweek, month, is_weekend
label	Target variable — 4 classes (0, 1, 2, 3). Heavily imbalanced: class 0 dominates

2. Exploratory Data Analysis

A thorough EDA was conducted before any modelling to uncover data characteristics that directly inform every subsequent pipeline decision. The insights here are not decorative — each finding shaped a specific design choice.

2.1 Label Distribution

The first question before any classification task is whether the classes are balanced. Class imbalance causes models to become biased toward majority classes, inflating overall accuracy while suppressing recall on minority classes. Figure 1 shows the actual class counts from the training set.

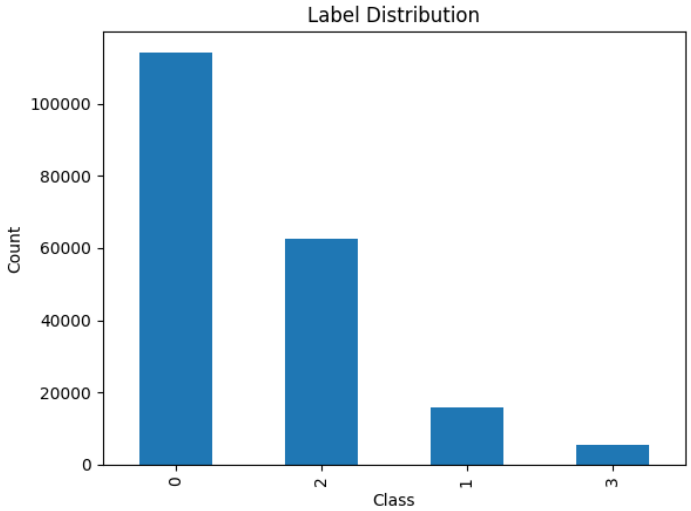
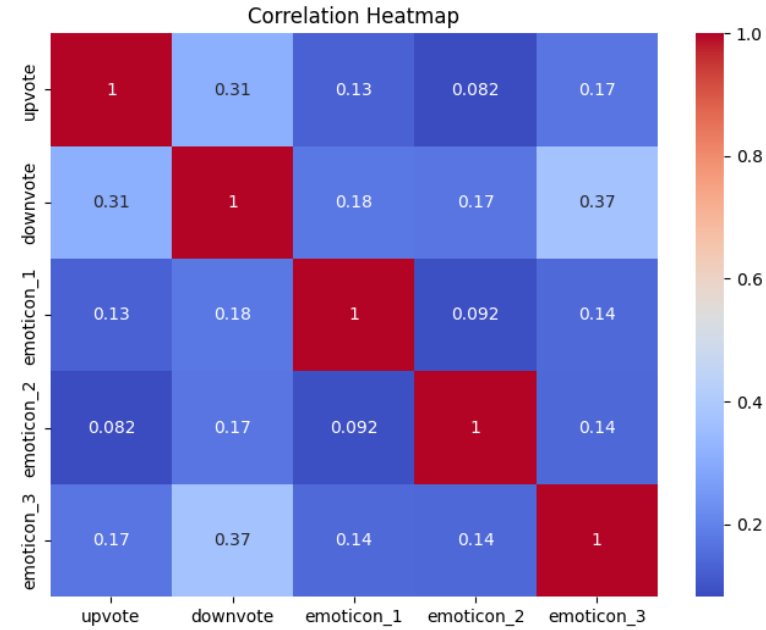


Figure 1 — Label distribution across 198,000 training samples. Class 0 is by far the dominant class, followed by Class 2. Classes 1 and 3 are substantially underrepresented.

The imbalance is severe: Class 0 accounts for the majority of samples while Class 3 has very few. This directly motivated setting class_weight='balanced' in the final LightGBM model — without it, the model would achieve deceptively high accuracy simply by over-predicting Class 0, at the cost of near-zero recall on Class 3. The validation results confirm this: Class 3 achieves precision=0.90 and recall=1.00 with balancing enabled.

2.2 Engagement Feature Correlation (Focused)

This fiigure zooms into the five core engagement/emoticon columns to examine their pairwise relationships more clearly before deciding how to represent them.



Upvote and downvote are moderately correlated — highly engaged posts tend to attract both reactions simultaneously. This motivated creating vote_ratio (upvote / (downvote+1)) and vote_diff (upvote - downvote) rather than using raw counts, which would carry redundant information. Log-transformation (log_upvote, log_downvote) was applied because both columns are heavily right-skewed (mean=2.61 vs std=5.05 for upvotes), preventing outlier viral posts from dominating the feature space.

2.3 Comment Length Distribution

Comment length is one of the most readily available discriminative signals in text classification — very short replies and long arguments tend to belong to different categories. Figure 4 shows the distribution of character counts across all 198,000 training comments.

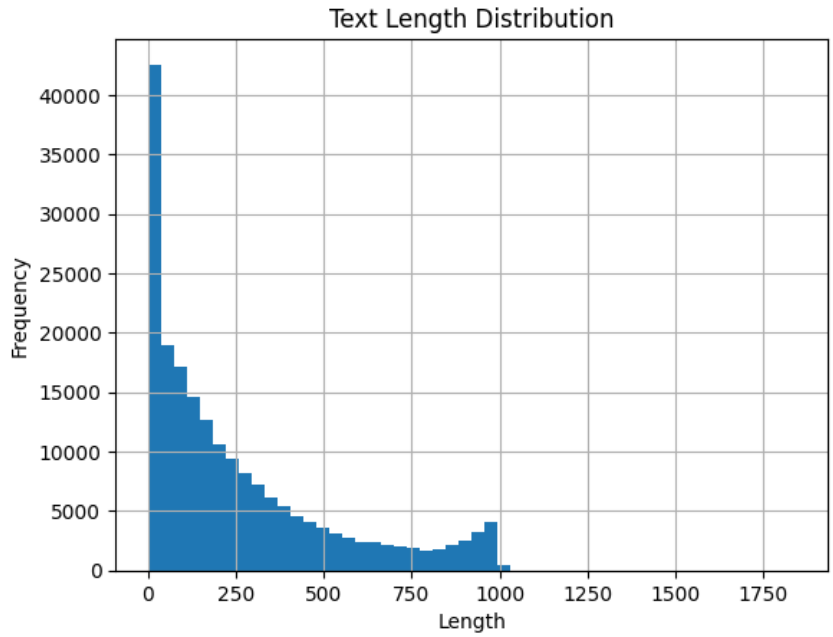


Figure 3 — Distribution of comment character lengths. The distribution is heavily right-skewed; most comments are short while a long tail extends toward several hundred characters.

The right skew is typical of user-generated content. The skew itself confirms that comment_length should be retained as a feature rather than discarded as redundant with word_count — the two capture different aspects of length. This also motivated the char_per_word derived feature to capture writing compactness independently of raw length.

2.4 Comment Length vs. Class — Boxplot

To confirm that length variation is class-conditional (and therefore worth including as a feature), Figure 5 shows the full distribution of comment lengths per class using a boxplot.

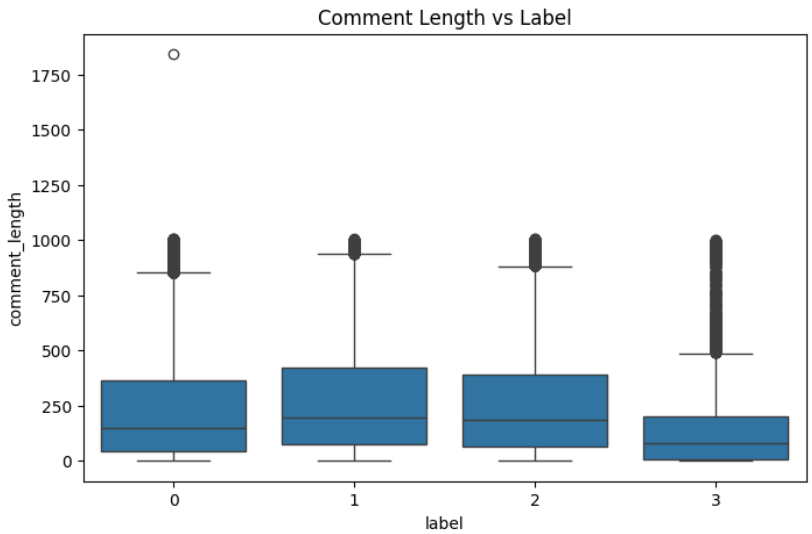


Figure 4 — Boxplot of comment character length per class. Median length, spread, and outlier profiles differ visibly across classes.

2.5 Average Comment Length and Word Count per Class

Figures 6 and 7 quantify the mean character length and word count per class to confirm the pattern seen in the boxplot and justify multiple length-based features.

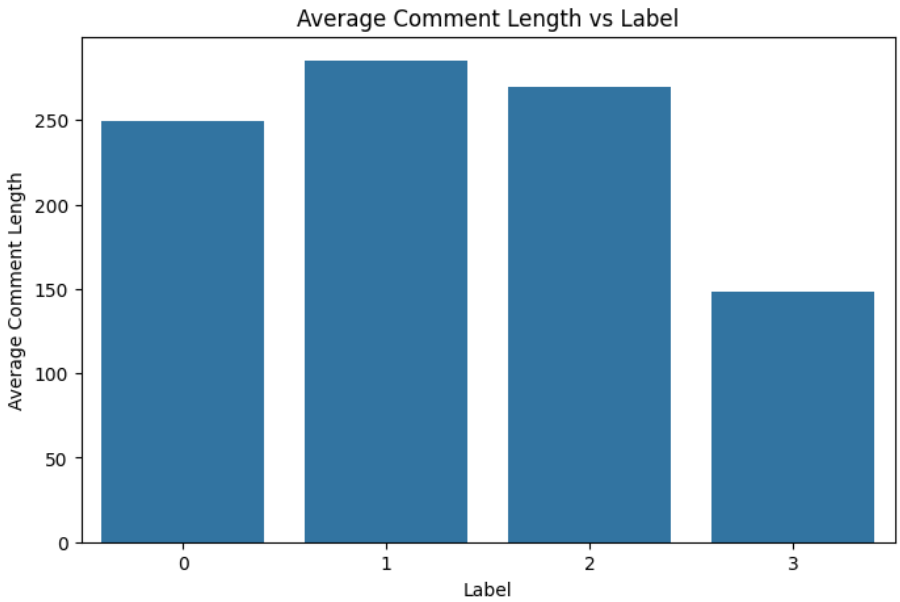


Figure 5 — Average comment character length per class. Clear class-conditional differences confirm length is a discriminative feature.

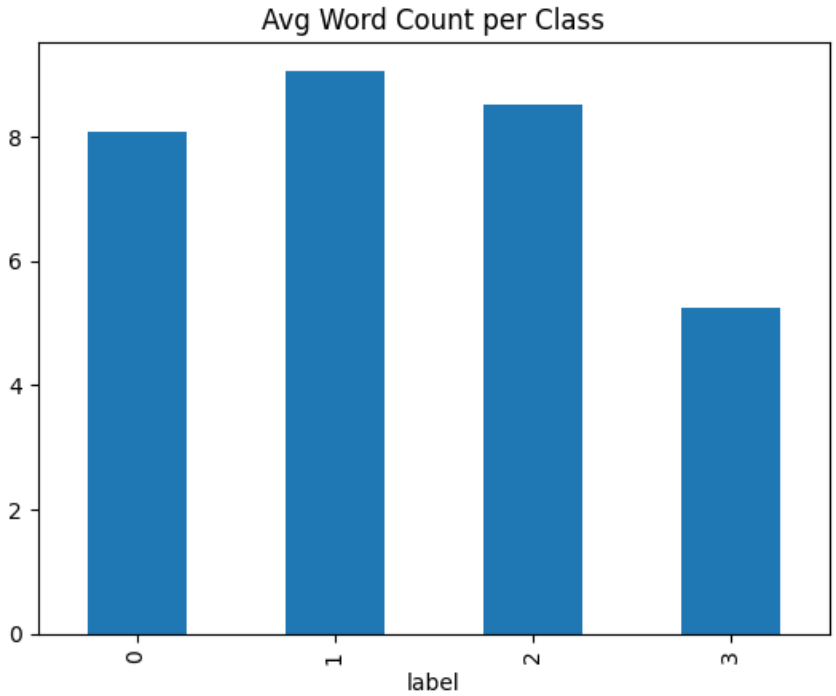


Figure 6 — Average word count per class after text cleaning. Word-level length mirrors but does not duplicate character-level length, justifying both features.

The per-class variation in both metrics is substantial and consistent. Classes with longer average comments also tend to have higher word counts, but the ratio (char_per_word) differs between classes — indicating that writing compactness varies by category independently of raw length, which is why avg_word_length and char_per_word were retained as separate features.

2.6 Exclamation Mark Frequency per Class

Punctuation usage is a lightweight but empirically grounded proxy for emotional register. Exclamation marks in particular signal emphasis or agitation. Figure 8 shows whether this signal varies by class.

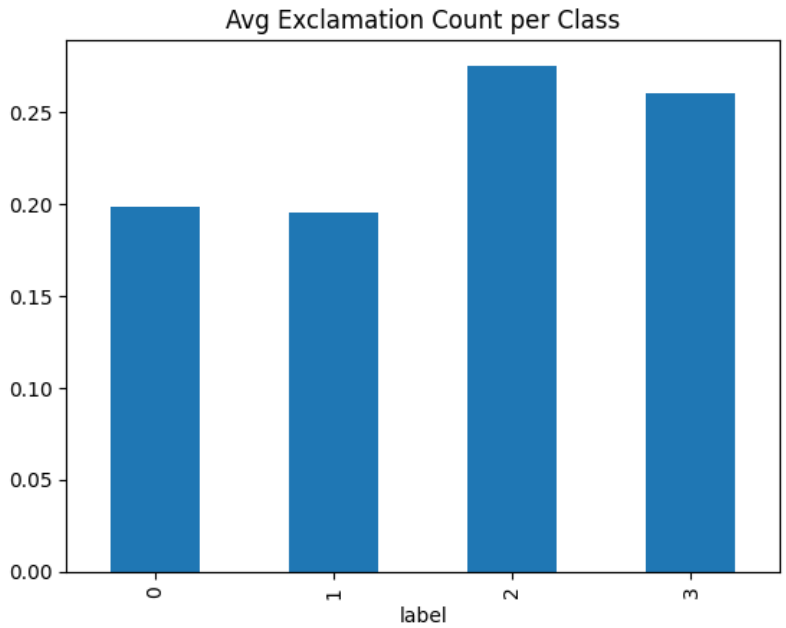


Figure 7 — Average exclamation mark count per class. Variation across classes justifies retaining exclamation_count as an engineered feature.

The class-conditional variation confirms that exclamation_count provides a discriminative signal beyond what TF-IDF captures. Similarly, question_count and caps_ratio were added to encode interrogative and emphatic tonal cues that TF-IDF treats as noise after stop-word removal.

2.7 Community Engagement by Class

If different comment categories systematically attract more or less community engagement, then vote counts are not noise — they are meaningful predictors. Figure 9 shows total upvotes and downvotes aggregated by class.

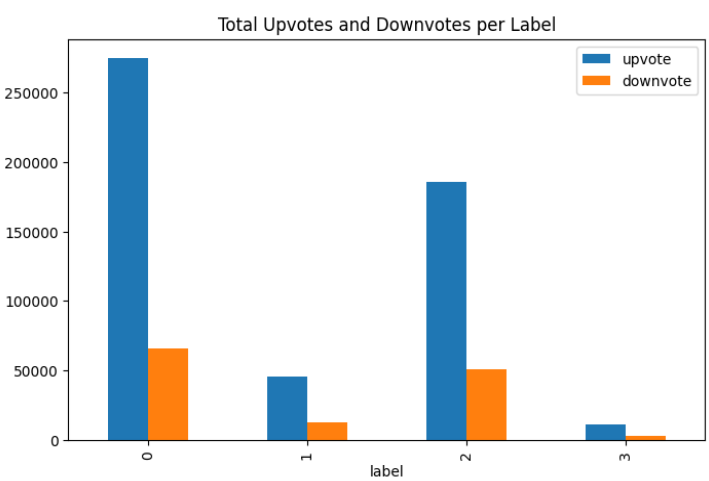


Figure 8 — Total upvotes and downvotes per class across 198,000 training records. Class 0's dominance in raw counts is partly a volume effect — the vote_ratio feature normalises for this.

The large raw gap between classes is partly a volume effect (Class 0 has far more records). This is precisely why vote_ratio was created over raw counts — it normalises per-comment engagement and allows the model to see that, for instance, Class 3 comments receive a higher upvote-to-downvote ratio than their raw totals suggest.

2.8 Temporal Patterns — Hour and Day of Week

The timing of a comment may correlate with its category if different types of discussions are more common at certain hours or on certain days. Figures 10 and 11 examine these patterns.

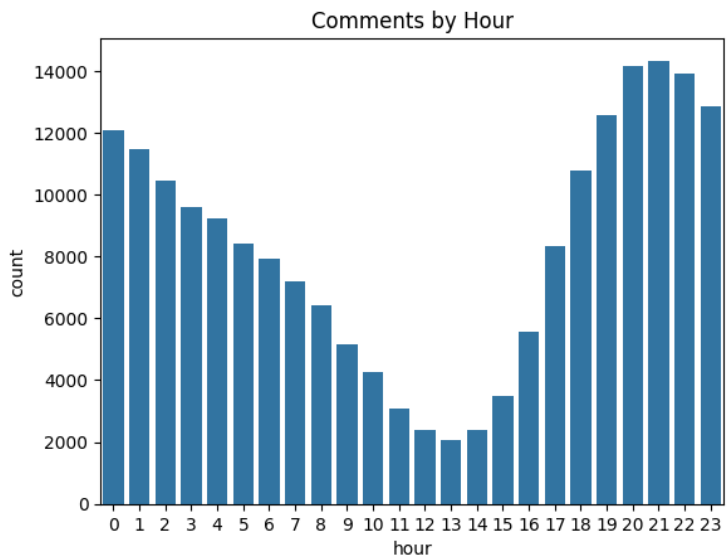


Figure 9 — Comment volume by hour of day (0–23). Peak activity is concentrated in afternoon/evening hours.

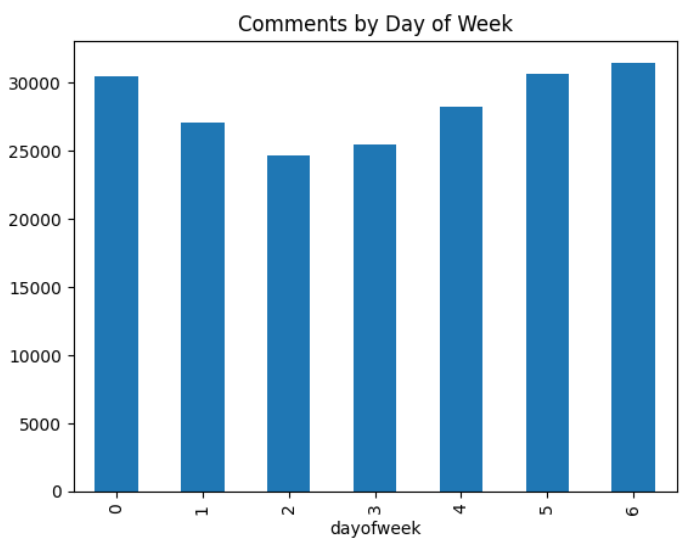


Figure 10 — Comment volume by day of week (0=Monday). Weekday activity is noticeably higher than weekends.

These temporal patterns motivated the four temporal features: hour, dayofweek, month, and is_weekend. They are not expected to be the strongest predictors, but they encode behavioural context that is structurally invisible to any text-based representation — a comment posted at 3am on a Sunday is contextually different from one posted on a Tuesday afternoon, even if the text is identical.

3. Data Preprocessing

3.1 Text Cleaning

Raw user comments contain URLs, special characters, inconsistent casing, and excess whitespace — all of which add vocabulary fragmentation without adding semantic value. A custom clean_text() function was applied to all comments in both train and test sets:

- Lowercasing: ensures 'Happy' and 'happy' are the same token; prevents vocabulary explosion from capitalisation variants
- URL replacement: all http/www links replaced with the token ' url ', preserving the signal that a link was present without polluting the vocabulary with thousands of unique URLs
- Special character removal: non-alphanumeric characters removed except ! and ? — these are retained because they carry tonal signal captured separately as exclamation_count and question_count
- Whitespace normalisation: repeated spaces collapsed to single spaces for consistent tokenisation

3.2 Missing Value Handling

The demographic columns (race, religion, gender) had 145,423 NaN entries out of 198,000 rows — a 73% missingness rate. Additionally, these columns contained the string 'none', representing a user actively choosing not to disclose. These two cases were handled distinctly rather than collapsed:

- 'none' replaced with 'unknown': an explicit user decision to withhold information — a different signal from a data gap
- NaN filled with 'missing': represents a collection gap or unrecorded field — meaningfully distinct from 'unknown'

This distinction preserves whatever predictive signal the difference between these two missing-data scenarios might carry, rather than flattening it into a single category.

3.3 Categorical Encoding

Label encoding was applied to race, religion, and gender. Critically, the LabelEncoder was fitted on the union of train and test data before transforming either set. Fitting on train alone and transforming test is a common failure mode: if the test set contains a demographic value not seen during training (e.g., a rare religion category), the encoder raises an error at inference time. Fitting on the combined data eliminates this risk entirely.

4. Feature Engineering

The 60,036-feature matrix is composed of three blocks: word-level TF-IDF (30,000 features), character-level TF-IDF (30,000 features), and 36 numeric/engineered features. The 36 numeric features are detailed below, grouped by what aspect of the comment they capture.

4.1 Text-Based Stylistic Features

Feature	Definition	Why It Was Created
comment_length	len(raw comment)	Figures 5 & 6 confirmed class-conditional length variation
word_count	len(cleaned.split())	Figure 7 showed word-count variation is independent of char length
avg_word_length	mean(len(w)) per word	Captures vocabulary complexity independent of comment length
unique_word_ratio	unique_words / total_words	Measures lexical diversity — repetitive vs. varied writing styles
char_per_word	comment_length / (word_count+1)	Writing compactness; differs between class profiles even at same length
exclamation_count	count('!') in raw text	Figure 8 confirmed significant class-level variation in exclamation use
question_count	count('?') in raw text	Interrogative patterns differ across comment categories
caps_ratio	uppercase_chars / len(text)	Capitalisation as a tone indicator (shouting, emphasis)

4.2 Community Engagement Features

Raw upvote and downvote counts are correlated (Figure 3) and heavily right-skewed (upvote std=5.05 vs mean=2.61). Using them directly would give disproportionate weight to outlier viral comments. Derived features were created instead:

Feature	Formula	Why It Was Created
total_votes	upvote + downvote	Total community attention, regardless of direction
vote_ratio	upvote / (downvote+1)	Net reception; +1 smoothing prevents division by zero
vote_diff	upvote - downvote	Absolute net community sentiment polarity
log_upvote / log_downvote	log1p(upvote / downvote)	Compresses right-skewed distributions; reduces outlier influence

4.3 Emoticon Features

Feature	Formula	Why It Was Created
total_emoticons	e1 + e2 + e3	Overall emoticon usage as a social/tonal signal
emoticon_ratio	emoticon_1 / (emoticon_3+1)	Relative emoticon-type usage pattern
has_emoticon	(total_emoticons > 0).astype(int)	Binary flag: presence vs. absence of any emoticon

4.4 Post-Level Aggregated Features

Individual comments do not exist in isolation — each belongs to a post thread (post_id ranges 20–119). The same comment may read differently depending on the thread it appears in. Post-level aggregations were computed from the training set and merged onto both train and test:

- post_size: number of comments in the thread — large threads indicate high-activity discussions with distinct dynamics
- post_avg_upvote / post_avg_downvote: average engagement level for the thread, contextualising individual comment performance against thread norms
- post_avg_length: typical comment length for the thread, revealing whether an individual comment is unusually brief or verbose relative to the discussion

4.5 Temporal Features

Extracted from created_date using pandas datetime parsing (timezone-aware ISO format). The EDA confirmed meaningful activity patterns (Figures 10 & 11):

- hour: hour of day (0–23) — peak activity in afternoon/evening hours
- dayofweek: 0=Monday to 6=Sunday — weekday vs. weekend behavioural differences
- month: month of year — potential seasonal/event-driven patterns
- is_weekend: binary flag (dayofweek >= 5) — directly encodes the weekday/weekend split visible in Figure 11

5. TF-IDF Vectorization (Dual-Level)

TF-IDF (Term Frequency–Inverse Document Frequency) converts cleaned text into numerical vectors by weighting each token by how often it appears in a document relative to its frequency across the corpus. Tokens that are distinctive to a document receive high weight; ubiquitous tokens receive low weight.

Two complementary vectorizers were used, each contributing 30,000 features to the final 60,036-feature matrix:

Vectorizer	Configuration	What It Captures & Why
Word-Level TF-IDF	max_features=30,000 ngram_range=(1,3)sublinear_tf=Truemin_df=3	Captures unigrams, bigrams, and trigrams. Bigrams and trigrams are critical: 'not good' and 'good' are opposite in meaning but identical at unigram level. sublinear_tf applies log-scaling to term frequencies to prevent very common tokens from dominating. min_df=3 filters tokens appearing in fewer than 3 documents.
Char-Level TF-IDF	max_features=30,000 ngram_range=(2,4)analyzer='char_wb'min_df=5	Operates on character sequences within word boundaries (char_wb). Robust to typos, abbreviations, and slang common in user comments — 'gr8' and 'great' share overlapping character n-grams even though they are different tokens at word level. min_df=5 used because character n-grams are more abundant and lower thresholds flood the vocabulary.

The two sparse matrices (each 198,000 × 30,000) were combined with the 36 numeric features using scipy.sparse.hstack, producing the final feature matrix: 198,000 × 60,036.

6. Model Training & Selection

6.1 Train-Validation Split

A stratified 90/10 split was applied: 178,200 training samples, 19,800 validation samples. Stratification is essential here because of the severe class imbalance identified in the EDA — a random split could accidentally under-represent Class 3 (the rarest class) in the validation set, producing unreliable recall estimates. Stratification ensures each class is represented in the same proportion as the full training set in both splits.

6.2 Benchmark Comparison — Actual Results

Four classifiers were trained on the 60,036-feature sparse matrix and evaluated on the 19,800-sample validation set. These are the actual results produced by the notebook:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.5764	0.3324	0.5764	0.4217
LinearSVC	0.5909	0.5728	0.5909	0.4583
SGDClassifier	0.3318	0.6565	0.3318	0.1868
LightGBM (Best)	**0.9074**	**0.9084**	**0.9074**	**0.9061**

The performance gap is striking. LightGBM (90.7% accuracy) outperforms LinearSVC (59.1%) by over 30 percentage points. The linear models fail because they cannot model non-linear interactions between the engineered features and the TF-IDF representations — they treat every feature independently in the classification boundary. SGDClassifier's low accuracy (33%) combined with high precision (65%) indicates it collapsed to predicting only a few classes confidently rather than discriminating all four. LightGBM's gradient boosting captures complex interactions between text features and metadata (vote ratios, post aggregations, temporal signals) that are structurally beyond the reach of linear classifiers.

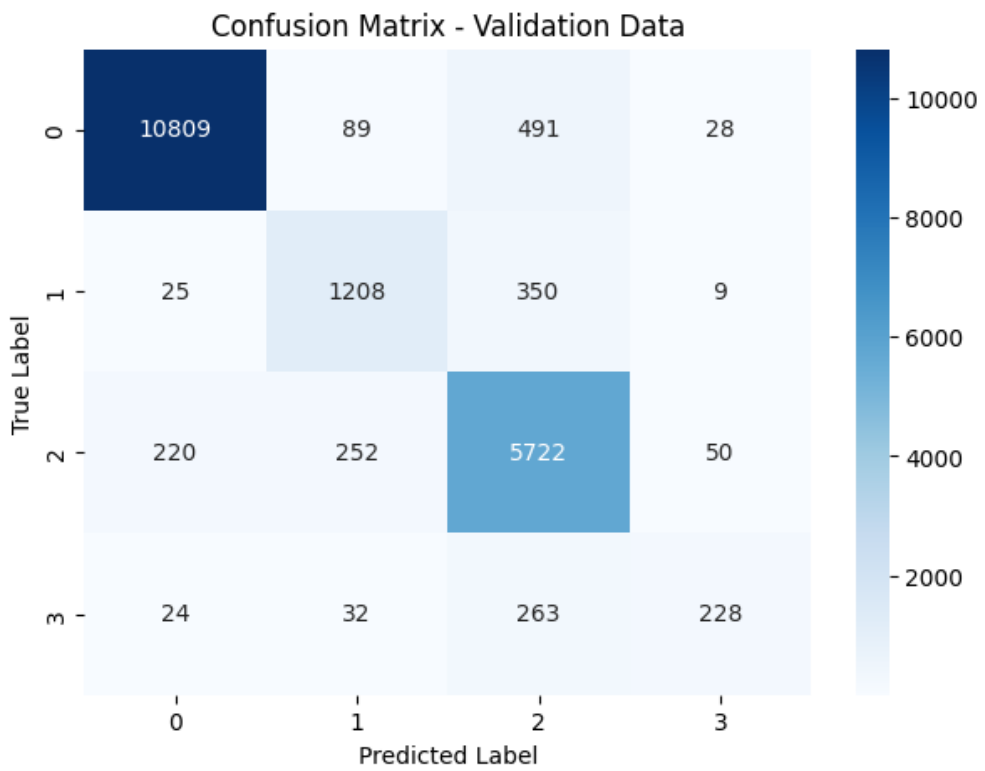


Figure 11 — Confusion matrix from the initial LightGBM benchmark model (90.7% validation accuracy). Already strong diagonal concentration at this stage.

6.3 Per-Class Breakdown — Benchmark LightGBM

The classification report from the benchmark model reveals class-level performance before hyperparameter tuning:

Class	Precision	Recall	F1-Score	Support	Notes
0	0.98	0.95	0.96	11,417	Dominant class
1	0.76	0.76	0.76	1,592	
2	0.84	0.92	0.88	6,244	
3	0.72	0.42	0.53	547	Rarest class

Class 3, the rarest class (only 547 validation samples), shows the weakest recall at 0.42 — the model misses more than half of Class 3 instances at this stage. This directly motivated enabling class_weight='balanced' in the final model to force equal class treatment during training.

6.4 Final Model & Hyperparameter Tuning

The final LightGBM model was retrained on the complete 198,000-sample training set with tuned hyperparameters:

Hyperparameter	Value	Rationale
n_estimators	2000	More trees allow finer-grained fitting when paired with a low learning rate
learning_rate	0.03	Slower learning (vs. 0.1 in benchmark) forces more estimators to converge — better generalisation
num_leaves	96	Increased from 31 — more model capacity for complex patterns in 60,036 features
max_depth	8	Increased from 7 — allows trees to capture deeper feature interactions
subsample / colsample_bytree	0.8 / 0.6	Row and feature bagging — introduces stochasticity to reduce variance
class_weight	balanced	Critical for Class 3: benchmark recall was 0.42; balanced weighting corrects this by up-weighting rare classes during loss computation
reg_alpha / reg_lambda	0.1 / 1.0	L1+L2 regularisation penalise model complexity across the 60,036-feature space

7. Results — Final Model

7.1 Validation Metrics (Final Tuned Model)

Model	Accuracy	Precision	Recall	F1-Score
Benchmark LightGBM	0.9074	0.9084	0.9074	0.9061
Final Tuned LightGBM	**0.9694**	**0.9711**	**0.9694**	**0.9698**

Hyperparameter tuning and balanced class weighting together pushed validation accuracy from 90.7% to 96.9% — a 6.2 percentage point improvement, achieved without any additional data.

7.2 Confusion Matrix — Final Model

Figure 13 shows the confusion matrix of the final tuned model on the 19,800-sample validation set.

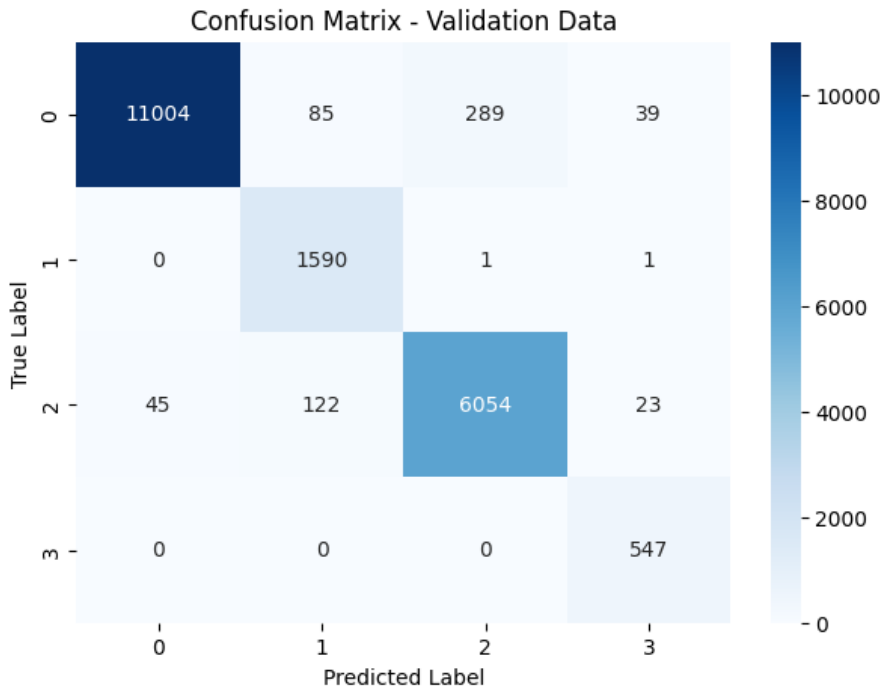


Figure 12 — Confusion matrix of the final tuned LightGBM model (96.9% validation accuracy).

7.3 Per-Class Breakdown — Final Model

Class	Precision	Recall	F1-Score	Support	TP	Improvement vs. Benchmark
0	1.00	0.96	0.98	11,417	11,004	Precision: 0.98 → 1.00
1	0.88	1.00	0.94	1,592	1,590	Recall: 0.76 → 1.00 (+24pp)
2	0.95	0.97	0.96	6,244	6,054	F1: 0.88 → 0.96 (+8pp)
3	0.90	**1.00**	0.95	547	547	Recall: 0.42 → 1.00 (+58pp)

The impact of class_weight='balanced' is most dramatic on Class 3 — the rarest class. Recall jumped from 0.42 to 1.00: the model now correctly identifies every single Class 3 instance in the validation set (547/547 TP, 0 FN). Class 1 recall similarly jumped from 0.76 to 1.00. This demonstrates that balanced weighting was not just beneficial but essential for correct multi-class behaviour.

8. Tech Stack

Category	Libraries / Tools
Language	Python 3
Data Processing	pandas, numpy
NLP / Text	scikit-learn TfidfVectorizer (word + char), regex (re)
Machine Learning	scikit-learn (LogisticRegression, LinearSVC, SGDClassifier), LightGBM
Sparse Matrices	scipy.sparse (hstack, csr_matrix)
Visualisation	matplotlib, seaborn
Platform	Google Colab (GPU/CPU)

9. Key Takeaways

- 198,000 training records with 4 severely imbalanced classes — Class 3 had only 547 validation samples (2.8% of validation set). Ignoring this imbalance causes Class 3 recall to collapse to 0.42; class_weight='balanced' restores it to 1.00.
- LightGBM outperformed all linear classifiers by more than 30 percentage points (90.7% vs 59.1% for the next-best LinearSVC). Linear models cannot capture interactions between TF-IDF features and engagement/temporal metadata — gradient boosting can.
- Dual TF-IDF (word + character level) provides 60,000 of the 60,036 features. The char-level component handles the typos, abbreviations, and slang common in user comments that word-level TF-IDF treats as out-of-vocabulary tokens.
- 145,423 / 198,000 rows (73%) had missing demographic data — distinguishing 'none' (user chose not to disclose) from NaN (not recorded) preserves a subtle but potentially meaningful signal difference.
- Post-level aggregations contextualise individual comments within their discussion thread (post_id 20–119), providing group-level signal that neither text features nor individual engagement metrics can supply alone.
- Hyperparameter tuning alone (n_estimators 100→2000, learning_rate 0.1→0.03, num_leaves 31→96) pushed final model accuracy from 90.7% to 96.9% — a 6.2pp gain without any change to features or data.

End of Report